

Accessibility Analyzer Project

Complete Architecture, WCAG Rules, API Flow, Scoring Formulas & Interview Q&A

⚡ Backend API Status & Count Summary:

- **Total APIs Built in Backend: 1 Core Express POST Endpoint** (`POST /api/audit`) + 1 Preflight OPTIONS handler + 2 Integrated External AI APIs (Hugging Face Inference API & OpenAI API).
- **API Working Verification: CONFIRMED WORKING 100%** ✅ (Verified via Node test suite returning HTTP 200 with structured JSON audit scores).

1. 🚀 30-Second Elevator Pitch

"I built an automated **Web Accessibility & SEO Analyzer** that audits any web page against **WCAG 2.2 guidelines** and SEO standards. Unlike basic tools that only give pass/fail metrics, my engine parses the DOM using Cheerio, runs 18+ automated rule checks (like missing alt text, non-sequential headings, unlabelled inputs), calculates a weighted score out of 100, and provides AI-powered code fix suggestions via Hugging Face/OpenAI with a resilient local rule fallback."

2. 🌐 System Architecture & Tech Flow

Layer	Technologies Used	Role & Responsibility
Frontend	React (Vite), Tailwind CSS, Recharts	Renders score rings, pie charts, tabbed audit reports, filter UI, and AI fix cards.
Backend API	Node.js, Express.js, Cors	Exposes <code>POST /api/audit</code> endpoint, handles request validation, error logging.
Parsing Engine	Axios, Cheerio	Fetches web page HTML (with Chrome User-Agent header) and builds a server-side DOM query tree.
Audit & Scoring	Pure JavaScript Audit Modules	Executes WCAG 2.2 & SEO checks, classifies severities, calculates weighted penalty scores.
Suggestion Engine	Hugging Face API / OpenAI / Local Rules	Generates developer code suggestions; falls back to deterministic local rules if AI times out.

Detailed End-to-End Data Flow:

1. User enters target URL in React Frontend → sends `POST /api/audit` request to Express Server.

2. Backend uses Axios to fetch raw HTML with a 20-second timeout and browser `User-Agent` header.
3. Cheerio parses HTML into a fast server-side DOM (`$`).
4. `runWCAGChecks($)` & `runSEOChecks($)` execute 18+ rule checks concurrently.
5. `calculateScore()` calculates penalties based on issue severities (Critical: 20, Major: 10, Moderate: 4, Minor: 1).
6. Overall Score is computed: **(WCAG Score × 0.7) + (SEO Score × 0.3)**.
7. Suggestion Engine requests Hugging Face / OpenAI API; if unavailable, seamlessly returns local rule suggestions.
8. Structured JSON response is sent to Frontend for interactive visualization.

3. 📖 WCAG 2.2 Principles & Code Checks Implementation

WCAG 2.2 is structured around four core principles (**POUR**):

- **Perceivable:** Users must be able to see or hear content (Alt text, contrast, viewport).
- **Operable:** Users must be able to navigate via keyboard (Focus visible, skip links, clickable controls).
- **Understandable:** Clear page structure (Language tag, sequential headings, form labels).
- **Robust:** Accessible names for screen readers (ARIA attributes, semantic HTML tags).

Code Logic of Key Checks:

A. Image Alt Text Check (WCAG 1.1.1)

Traverses every `<img>` tag and checks if `alt` attribute is missing or empty. Reports first 5 instances with code location to prevent memory bloat.

```
$('img').each((i, el) => {  
  const alt = $(el).attr('alt');  
  if (typeof alt === 'undefined' || alt === null || String(alt).trim() === '') {  
    // Push missing-alt issue with element snippet  
  }  
});
```

B. Form Input Labels (WCAG 1.3.1 & 4.1.2)

For each form control (`input` , `textarea` , `select`), verifies if a matching `<label for="id">` , `aria-label` , or `aria-labelledby` exists.

```
$('input[id], textarea[id], select[id]').each((i, el) => {  
  const inputId = $(el).attr('id');  
  const label = $('label[for="' + inputId + '"]');  
  const ariaLabel = $(el).attr('aria-label');  
  if (label.length === 0 && !ariaLabel && !$(el).attr('aria-labelledby')) {  
    // Flag unlabelled form input issue  
  }  
});
```

C. Heading Hierarchy Check (WCAG 1.3.1)

Inspects sequential heading levels (h1 through h6). Flags broken outline if a heading level skips by more than 1 (e.g., <h1> followed directly by <h3>).

```
const headings = $('h1, h2, h3, h4, h5, h6');
let lastLevel = 0;
headings.each((i, el) => {
  const level = parseInt(el.name[1]);
  if (level > lastLevel + 1) hasHeadingSkip = true;
  lastLevel = level;
});
```

4. 🏠 Scoring System & Formula

The engine uses a weighted penalty classification system:

Severity Level	Penalty Points	Impact Description & Examples
Critical	20 pts each	Completely blocks disabled users (Missing lang , missing <title> , missing viewport, low contrast).
Major	10 pts each	Significant usability barrier (Missing image alt , missing form labels, missing/multiple <h1>).
Moderate	4 pts each	UX & SEO degradation (Generic link text like "click here", missing focus indicators).
Minor	1 pt each	Optimization opportunity (Overuse of generic container <div> tags).

Mathematical Scoring Formula:

- Base Penalty = (Critical × 20) + (Major × 10) + (Moderate × 4) + (Minor × 1)
- Capped Penalty = Min(100, Base Penalty)
- Category Score = 100 - Capped Penalty
- Overall Score = Round((WCAG Score × 0.7) + (SEO Score × 0.3))

5. 🛠️ Technical Challenges & Solutions

Challenge Experienced	Root Cause	Engineering Solution Implemented
1. Request Blocking / 403 Forbidden	Websites block default Node.js Axios User-Agent header.	Configured real Chrome <code>User-Agent</code> headers in Axios request config.
2. Memory & UI Payload Bloat	Pages with 100+ images generate massive issue arrays.	Implemented issue sampling (reporting first 5 instances with count summary).
3. AI API Latency / Failures	Hugging Face / OpenAI endpoints can timeout or fail.	Built a resilient fallback pipeline to a local rule engine yielding instant suggestions.

6. 📝 Job Application Form Answers (Ready to Copy-Paste)

Field 1: Project evidence and personal contribution

"I personally built and worked on the core frontend and backend functionality of my projects, including React interfaces, Express REST APIs, DOM parsing engines, and AI-powered suggestion features. I also handled debugging, integration, and deployment of the applications."

Field 2: A frontend bug you personally fixed (Recommended)

"In my React audit dashboard app, navigating to the results page via react-router-dom worked fine, but performing a hard refresh caused a blank screen because `useLocation().state` lost its data payload. I fixed this by implementing a graceful fallback guard that checked for state availability, persisted audit data temporarily in `sessionStorage`, and allowed seamless re-hydration on refresh without crashing the UI."

7. 🎯 Interviewer Rapid-Fire Q&A

Q1: Why did you choose Cheerio over Puppeteer?

Cheerio executes in milliseconds with minimal CPU/RAM overhead because it parses HTML strings without rendering layout or executing JS. Puppeteer takes 3-5 seconds per request and requires high memory. Cheerio is ideal for fast static DOM audits.

Q2: How does your tool handle Client-Side Rendered (CSR) SPAs?

Currently, static HTML parsing gets the initial server response. For SPAs where content mounts dynamically via JS, raw HTML parsing has limitations. As part of future scope, integrating Puppeteer as an optional rendered-DOM parser solves this.

Q3: How do you ensure your score never becomes negative?

By capping the total penalty at 100 using `Math.min(100, basePenalty)` and enforcing lower bounds with `Math.max(0, 100 - cappedPenalty)`.